

Suralis — Sistema de Reportes Ambientales

1. Descripción General

Suralis es una aplicación móvil para el reporte de incidencias ambientales (basura, escombros, contaminación de aguas y otros). Los usuarios pueden fotografiar una incidencia, registrar su ubicación GPS y enviar el reporte. El mapa principal muestra todos los reportes activos con marcadores coloreados según su estado. Otros usuarios pueden confirmar reportes ("Yo también lo vi") y dejar comentarios. La app soporta modo invitado para explorar el mapa sin registro.

2. Equipo

Nombre	Rol	Responsabilidades
Bastian Contreras	Líder + Backend	Arquitectura, revisión de PRs, Docker, estructura API
Mathias	Backend	Auth: register, login, logout, Sanctum
Sebastian	Backend	Migraciones, modelos, CRUD reportes, heatmap, seeder
Alonso	Frontend	Pantallas Login/Registro, navegación, token
Catalina	Frontend	MapScreen, marcadores, heatmap visual, detalle reporte
Oskar	Frontend	Formulario nuevo reporte, cámara, GPS, envío al backend

3. Arquitectura General



La API es un backend JSON puro — sin vistas Blade ni sesiones. Todos los endpoints viven bajo `/api/*`. La autenticación usa Laravel Sanctum con token Bearer en el header `Authorization`.

Repositorio

Proyecto-reportes/	
├─ backend/	← Laravel 12 REST API
├─ frontend/	← React Native (Expo)
└─ docs/	← Documentación y contrato API

4. Base de Datos

Diagrama de tablas

```
users
├─ id (PK)
├─ name
├─ email (unique)
├─ password
└─ timestamps

categories
├─ id (PK)
├─ name → "basura" | "escombros" | "aguas" | "otro"
└─ timestamps

reports
├─ id (PK)
├─ user_id (FK → users)
├─ category_id (FK → categories)
├─ description
├─ latitude (decimal 10,8)
├─ longitude (decimal 11,8)
├─ photo_path (nullable)
├─ status → "Pendiente" | "En Progreso" | "Resuelto"
└─ timestamps

confirmations
├─ id (PK)
├─ user_id (FK → users, cascade delete)
├─ report_id (FK → reports, cascade delete)
├─ UNIQUE (user_id, report_id)
└─ timestamps

comments
├─ id (PK)
├─ user_id (FK → users, cascade delete)
├─ report_id (FK → reports, cascade delete)
├─ body (text, max 500 chars)
└─ timestamps
```

Tabla **reports** — detalle de campos

Campo	Tipo	Descripción
id	bigint PK	Auto-incremental
user_id	bigint FK	Usuario que creó el reporte
category_id	bigint FK	Categoría (basura, escombros, etc.)
description	text	Descripción del problema

latitude	decimal(10,8)	Latitud GPS
longitude	decimal(11,8)	Longitud GPS
photo_path	varchar nullable	Ruta relativa en storage/public
status	enum	Pendiente / En Progreso / Resuelto
created_at	timestamp	Fecha de creación
updated_at	timestamp	Última actualización

Tabla `confirmations` — detalle de campos

Campo	Tipo	Descripción
<code>id</code>	bigint PK	Auto-incremental
<code>user_id</code>	bigint FK	Usuario que confirmó
<code>report_id</code>	bigint FK	Reporte confirmado
<code>created_at</code>	timestamp	Fecha de confirmación

La combinación `(user_id, report_id)` tiene restricción `UNIQUE` — un usuario solo puede confirmar un reporte una vez.

5. Configuración Inicial del Backend

Pre-requisitos

- Docker Desktop instalado y corriendo
- Git

Primera vez

```
# 1. Clonar el repositorio
git clone https://github.com/Bazzty/Proyecto-reportes.git
cd Proyecto-reportes

# 2. Configurar variables de entorno del backend
cp backend/.env.example backend/.env
# Editar backend/.env y rellenar:
#   DB_PASSWORD=tu_password
#   DB_ROOT_PASSWORD=tu_root_password

# 3. Construir e iniciar los contenedores
cd backend
docker compose up -d --build

# 4. Generar clave de la aplicación
docker compose exec app php artisan key:generate
```

```
# 5. Publicar archivos de Sanctum
docker compose exec app php artisan vendor:publish \
    --provider="Laravel\Sanctum\SanctumServiceProvider"

# 6. Ejecutar migraciones
docker compose exec app php artisan migrate

# 7. Crear enlace de almacenamiento público
docker compose exec app php artisan storage:link

# 8. (Opcional) Poblar con datos de prueba
docker compose exec app php artisan db:seed
```

Variables de entorno clave (backend/.env)

```
APP_NAME=Suralis
APP_ENV=local
APP_KEY=base64:... # generado por key:generate
APP_URL=http://localhost:8000

DB_CONNECTION=mysql
DB_HOST=db
DB_PORT=3306
DB_DATABASE=suralis
DB_USERNAME=suralis
DB_PASSWORD=tu_password
DB_ROOT_PASSWORD=tu_root_password
```

6. Comandos Docker diarios

```
# Entrar al directorio backend
cd backend

# Iniciar todos los servicios
docker compose up -d

# Detener todos los servicios
docker compose down

# Ver logs en tiempo real
docker compose logs -f app

# Ejecutar comando Artisan
docker compose exec app php artisan <comando>

# Acceder a shell del contenedor
docker compose exec app bash

# Ejecutar migraciones
```

```
docker compose exec app php artisan migrate

# Revertir última migración
docker compose exec app php artisan migrate:rollback

# Resetear BD y re-sembrar datos de prueba
docker compose exec app php artisan migrate:fresh --seed

# Crear nueva migración
docker compose exec app php artisan make:migration nombre_migracion

# Crear modelo + migración + controlador
docker compose exec app php artisan make:model NombreModelo -mc

# Ejecutar tests
docker compose exec app php artisan test

# Ejecutar un test específico
docker compose exec app php artisan test tests/Feature/AuthTest.php

# Ver logs de Laravel
docker compose exec app tail -f storage/logs/laravel.log
```

URLs de servicios

Servicio	URL
API REST	http://localhost:8000/api
phpMyAdmin	http://localhost:8080
Storage de imágenes	http://localhost:8000/storage/photos/

7. Configuración del Frontend

Pre-requisitos

- Node.js 18+ instalado localmente
- Expo Go instalado en el dispositivo móvil (iOS o Android)

Instalación

```
cd frontend
npm install
```

Variable de entorno

Crear o editar frontend/.env :

```
# IP local de la máquina donde corre Docker
```

```
EXP0_PUBLIC_API_URL=http://192.168.1.141:8000/api
```

Importante: La IP debe ser la de tu máquina en la red local, no `localhost`. El dispositivo móvil y la computadora deben estar en la misma red WiFi.

Para obtener tu IP local:

```
# macOS / Linux
ipconfig getifaddr en0

# Windows
ipconfig
```

Iniciar el servidor de desarrollo

```
cd frontend
npx expo start
```

Luego escanear el código QR con la app Expo Go desde el dispositivo.

8. Datos de Prueba (Seeders)

El seeder crea 4 usuarios y 21 reportes distribuidos alrededor del Lago Llanquihue.

Usuarios de prueba

Nombre	Email	Contraseña
Bastian Contreras	bastian@demo.cl	password
Catalina Muñoz	catalina@demo.cl	password
Oskar Pérez	oskar@demo.cl	password
Alonso Vera	alonso@demo.cl	password

Zonas de reportes

Zona	Cantidad
Puerto Varas (costanera)	5
Frutillar Bajo	4
Llanquihue (orilla norte)	3
Puerto Octay (extremo norte)	3
Ensenada (volcán Osorno)	2
Las Cascadas (orilla este)	2

Puerto Montt (acceso sur)	2
---------------------------	---

Ejecutar seeders

```
# Solo seeders (sin resetear migraciones)
docker compose exec app php artisan db:seed

# Reset completo + seeders
docker compose exec app php artisan migrate:fresh --seed
```

9. API — Endpoints

Autenticación

Método	Ruta	Auth	Descripción
POST	/api/register	✗	Registrar usuario
POST	/api/login	✗	Iniciar sesión
POST	/api/logout	✓	Cerrar sesión

POST /api/register

```
// Request
{
  "name": "Bastian Contreras",
  "email": "bastian@demo.cl",
  "password": "password",
  "password_confirmation": "password"
}

// Response 201
{
  "user": { "id": 1, "name": "Bastian Contreras", "email": "bastian@demo.cl" },
  "token": "1|aBcDeF..."
}
```

POST /api/login

```
// Request
{ "email": "bastian@demo.cl", "password": "password" }

// Response 200
{
  "user": { "id": 1, "name": "Bastian Contreras", "email": "bastian@demo.cl" },
  "token": "2|xYzAbC..."
}
```

Categorías

Método	Ruta	Auth	Descripción
GET	/api/categories	✗	Listar categorías

```
// Response 200
[
  { "id": 1, "name": "basura" },
  { "id": 2, "name": "escombros" },
  { "id": 3, "name": "aguas" },
  { "id": 4, "name": "otro" }
]
```

Reportes

Método	Ruta	Auth	Descripción
GET	/api/reports	✗	Listar todos los reportes
POST	/api/reports	✓	Crear nuevo reporte
GET	/api/reports/{id}	✗	Ver detalle de un reporte
GET	/api/reports/heatmap	✗	Puntos para el heatmap
GET	/api/user/reports	✓	Reportes del usuario autenticado

Estructura de un reporte (response)

```
{
  "id": 1,
  "description": "Acumulación de basura en la costanera",
  "latitude": -41.3198,
  "longitude": -72.9833,
  "photo_url": "http://localhost:8000/storage/photos/abc.jpg",
  "status": "Pendiente",
  "category": { "id": 1, "name": "basura" },
  "user": { "id": 1, "name": "Bastian Contreras" },
  "confirmations_count": 3,
  "confirmed_by_me": false,
  "created_at": "2026-06-29T12:00:00.000000Z"
}
```

POST /api/reports (multipart/form-data)

Campo	Tipo	Requerido
description	string	✓
latitude	numeric	✓

longitude	numeric	✓
category_id	integer	✓
photo	file (jpg/png/webp/gif, max 5MB)	✓

Confirmaciones

Método	Ruta	Auth	Descripción
POST	/api/reports/{id}/confirm	✓	Toggle confirmar/desconfirmar

```
// Response 200
{ "confirmed": true, "count": 4 }
```

Comentarios

Método	Ruta	Auth	Descripción
GET	/api/reports/{id}/comments	✗	Listar comentarios de un reporte
POST	/api/reports/{id}/comments	✓	Agregar comentario

```
// POST Request
{ "body": "Sigo viendo el mismo problema aquí." }

// Response 201
{
  "id": 1,
  "body": "Sigo viendo el mismo problema aquí.",
  "user": { "id": 2, "name": "Catalina Muñoz" },
  "created_at": "2026-06-29T14:30:00.000000Z"
}
```

10. Estructura del Backend

```
backend/
├── app/
│   ├── Http/
│   │   └── Controllers/
│   │       ├── AuthController.php      ← register, login, logout
│   │       ├── ReportController.php    ← CRUD reportes + heatmap
│   │       ├── CategoryController.php  ← lista categorías
│   │       ├── ConfirmationController.php ← toggle confirmación
│   │       └── CommentController.php   ← listar y crear comentarios
│   └── Models/
│       ├── User.php
│       ├── Report.php
│       └── Category.php
```

```

|   |   | Confirmation.php
|   |   | Comment.php
|   | database/
|   |   | migrations/
|   |   |   | ..._create_users_table.php
|   |   |   | ..._create_categorias_table.php
|   |   |   | ..._create_reports_table.php
|   |   |   | ..._create_confirmations_table.php
|   |   |   | ..._create_comments_table.php
|   |   | seeders/
|   |   |   | DatabaseSeeder.php
|   |   |   | CategorySeeder.php
|   |   |   | ReportSeeder.php
|   | routes/
|   |   | api.php                                ← definición de rutas
|   | docker-compose.yml

```

Flujo de una petición

1. El cliente envía `Authorization: Bearer <token>` en el header
2. Nginx recibe la petición en el puerto 8000 y la reenvía a PHP-FPM
3. Laravel verifica el token con Sanctum (`auth:sanctum` middleware)
4. El controlador procesa la lógica y retorna JSON
5. El cliente React Native recibe y renderiza los datos

11. Estructura del Frontend

```

frontend/
| App.js                                ← Navegación principal + restauración de sesión
| .env                                  ← EXPO_PUBLIC_API_URL
| src/
|   | screens/
|   |   | LoginScreen.js                ← Formulario de inicio de sesión
|   |   | RegisterScreen.js            ← Formulario de registro
|   |   | HomeScreen.js                ← Mapa principal con marcadores y filtros
|   |   | NewReportScreen.js           ← Crear nuevo reporte (foto + GPS)
|   |   | MyReportsScreen.js           ← Lista de reportes del usuario
|   |   | DetalleReporteScreen.js      ← Detalle, confirmaciones y comentarios
|   |   | MapScreen.js                 ← Mapa alternativo (componente Maps)
|   | components/
|   |   | Maps.js                      ← Componente de mapa con marcadores
|   | services/
|   |   | api.js                      ← Instancia axios configurada con Sanctum

```

Flujo de navegación





Gestión de sesión (App.js)

Al iniciar la app, se revisa `AsyncStorage` por `token` :

- Si existe → `setAuthToken(token)` y se navega a `Home`
- Si no → se navega a `Login`

El interceptor de axios detecta respuestas `401` y fuerza el logout automático.

```

// src/services/api.js
api.interceptors.response.use(
  (response) => response,
  async (error) => {
    if (error.response?.status === 401 && _onUnauthorized) {
      await AsyncStorage.removeItem('token');
      clearAuthToken();
      _onUnauthorized(); // redirige a Login
    }
    return Promise.reject(error);
  }
);

```

Modo Invitado

`LoginScreen` → "Continuar sin cuenta" → `Home({ guest: true })`

- Invitado puede: ver el mapa, ver reportes, leer comentarios
- Invitado NO puede: crear reportes, confirmar, comentar, ver "Mis Reportes"
- Al intentar una acción protegida → Alert con opción de ir al Login

Polling inteligente (HomeScreen)

```

useFocusEffect(
  useCallback(() => {
    let active = true;
    const fetchReports = async () => { /* ... */ };

    fetchReports();
    const interval = setInterval(fetchReports, 5000); // cada 5 segundos
    return () => { active = false; clearInterval(interval); }; // limpieza al
desmontar
  }, [isGuest])
);

```

La flag `active` previene actualizaciones de estado en componentes desmontados.

12. Funcionalidades Implementadas

Autenticación

- Registro con nombre, email y contraseña
- Login con token Sanctum persistido en AsyncStorage
- Logout que invalida el token en el servidor
- Restauración automática de sesión al reiniciar la app
- Redirección automática al login en tokens expirados (401)

Mapa (HomeScreen)

- Mapa centrado en Puerto Varas (Lago Llanquihue) como región inicial
- Marcadores con foto miniatura o ícono por categoría
- Borde del marcador coloreado según estado: amarillo (Pendiente), azul (En Progreso), verde (Resuelto)
- Callout al tocar un marcador: muestra foto, categoría, descripción, usuario y enlace "Ver detalle →"
- Filtros horizontales por categoría (chips seleccionables)
- Botón para centrar el mapa en la ubicación del usuario
- Heatmap de densidad en Android (`<Heatmap>` de react-native-maps)
- Saludo personalizado con nombre del usuario y contador de reportes propios
- UI optimista: reportes nuevos aparecen en el mapa inmediatamente antes de la confirmación del servidor
- Polling cada 5 segundos para mantener el mapa actualizado

Crear Reporte (NewReportScreen)

- Selección de foto desde cámara o galería
- Conversión automática a JPEG con compresión (resize a 1200px, calidad 0.75)
- Compatibilidad con formato HEIC (iPhone)
- Obtención de coordenadas GPS de alta precisión
- Selección de categoría mediante chips
- Envío como `multipart/form-data`

Mis Reportes (MyReportsScreen)

- Lista de reportes propios con foto a ancho completo
- Categoría con punto de color según estado
- Badge de estado (Pendiente / En Progreso / Resuelto)
- Coordenadas GPS y fecha de creación
- Tap en la tarjeta → navega a DetalleReporte
- Tap en la foto → lightbox pantalla completa con botón cerrar

Detalle de Reporte (DetalleReporteScreen)

- Foto en alta resolución
- Categoría, estado con color, descripción, nombre del autor y fecha
- Botón "Yo también lo vi" con contador de confirmaciones
 - Se rellena de turquesa al confirmar (toggle)
 - Oculto si el reporte es propio (solo muestra cuántos lo confirmaron)
 - Bloqueado para invitados (alert de login)
- Sección de comentarios con lista en orden cronológico inverso
 - Avatar circular con inicial del nombre
 - Input fijo con `KeyboardAvoidingView` para que el teclado no lo tape

- Invitados ven los comentarios pero no pueden escribir

Modo Invitado

- Botón "Continuar sin cuenta →" en la pantalla de Login
- Acceso de solo lectura al mapa, reportes y comentarios
- FAB y "Mis Reportes" deshabilitados visualmente (color gris)
- Alert con opción de ir al Login al intentar acciones protegidas

13. Cómo Probar la Aplicación

Flujo completo como usuario registrado

1. Abrir la app → pantalla de Login
2. Iniciar sesión con `bastian@demo.cl` / `password`
3. Ver el mapa con 21 reportes distribuidos por el Lago Llanquihue
4. Tocar un marcador → ver callout → tocar "Ver detalle →"
5. En el detalle, confirmar el reporte ("Yo también lo vi") y dejar un comentario
6. Volver al mapa, filtrar por categoría "basura"
7. Tocar el FAB (+) → crear un reporte con foto desde la cámara
8. Ver el nuevo reporte aparecer en el mapa inmediatamente
9. Ir a "Mis Reportes" → ver el reporte recién creado con su foto
10. Tocar la foto → ver lightbox pantalla completa

Flujo como invitado

1. Abrir la app → tocar "Continuar sin cuenta →"
2. Explorar el mapa y ver todos los reportes
3. Tocar un marcador → ver detalle → leer comentarios (sin poder escribir)
4. Tocar el FAB → aparece alert "Inicia sesión para crear un reporte"
5. Tocar "Iniciar sesión" en el alert → regresa a Login

Probar interacción entre usuarios

1. Iniciar sesión con `bastian@demo.cl`
2. Abrir un reporte de otro usuario y confirmar ("Yo también lo vi")
3. Dejar un comentario
4. Cerrar sesión → iniciar con `catalina@demo.cl`
5. Abrir el mismo reporte → ver la confirmación y comentario de Bastian
6. Agregar otro comentario → ambos aparecen en el detalle

14. Tecnologías Utilizadas

Backend

Tecnología	Versión	Uso
PHP	8.3	Lenguaje del backend
Laravel	12	Framework API REST
Laravel Sanctum	4.x	Autenticación por token
MySQL	8.0	Base de datos

Nginx	1.25	Servidor web (proxy)
Docker / Docker Compose	—	Contenedorización

Frontend

Tecnología	Versión	Uso
React Native	0.76	Framework móvil
Expo	52	Plataforma de desarrollo
React Navigation	7	Navegación entre pantallas
Axios	1.x	Cliente HTTP
react-native-maps	—	Mapa, marcadores, heatmap
expo-image	—	Imágenes con caché en disco
expo-image-picker	—	Cámara y galería
expo-image-manipulator	—	Compresión y conversión JPEG
expo-location	—	GPS
AsyncStorage	—	Persistencia local (token, usuario)

15. Flujo Git

```

main          ← estable, solo desde develop via PR
├─ develop ← integración, solo desde feat/* via PR
│   ├── feat/backend-auth
│   ├── feat/backend-reportes
│   ├── feat/CL-MapsMarkers
│   └─ feat/...
```

Convención de commits

```

feat:    nueva funcionalidad
fix:     corrección de bug
docs:    documentación
chore:   tareas de mantenimiento
refactor: refactorización sin cambio de comportamiento
```